Indexes in MS-SQL Server



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



Indexes in MS-SQL Server

-SQL Server clustered indexes

-SQL Server non-clustered indexes

Indexes in MS-SQL Server

In MS-SQL Server, an index is a database structure that improves the speed of data retrieval operations on database tables.

- It works like an index in a book, allowing the database engine to quickly locate and access specific data within a table. Indexes are created on one or more columns of a table and are used to optimize query performance.
- Indexes are special data structures associated with **tables** or **views** that help speed up the query.
- SQL Server provides two types of indexes: clustered index and non-clustered index.

Benefits of using Indexes

- **Enhanced Query Performance:** Indexes provide faster data retrieval by allowing the database engine to locate and access specific rows or ranges of rows quickly. With properly designed indexes, query execution time can be significantly reduced, resulting in improved overall performance.

- **Efficient Data Filtering:** Indexes enable efficient data filtering, especially when used on columns commonly used in search conditions or WHERE clauses. By creating indexes on frequently queried columns, the database engine can quickly narrow down the data set and retrieve the relevant rows.

- **Accelerated Join Operations:** Indexes on join columns improve the performance of join operations. They allow the database engine to efficiently locate matching rows in the joined tables, reducing the need for full table scans and improving query response time.

- **Sorted Data Retrieval:** Clustered indexes in MS-SQL Server determine the physical order of data in a table. This ensures that data retrieval is performed in a sorted manner when using the clustered index column(s). Sorted data retrieval can be particularly beneficial for queries with ORDER BY clauses.
- **Reduced Disk I/O:** Indexes reduce the amount of disk I/O required for query execution. Instead of scanning the entire table, the database engine can use index structures to quickly access the necessary data pages, minimizing disk read operations and improving overall query performance.
- **Minimized Locking and Blocking**: Well-designed indexes can reduce locking and blocking in multi-user environments. By providing efficient access paths to data, indexes enable concurrent data retrieval, reducing contention and enhancing system concurrency.

- **Improved Aggregation and Grouping:** Indexes can significantly improve the performance of aggregation and grouping operations. By creating indexes on columns used in GROUP BY clauses, the database engine can quickly locate and group the data, resulting in faster processing of such queries.
- **Optimized Joins and Foreign Key Constraints:** Indexes on join columns and foreign key columns enhance the performance of join operations and ensure efficient enforcement of referential integrity. They allow for quick lookups and matching of related rows, reducing the overhead of constraint validation and improving query execution speed.
- **Flexibility in Index Design:** MS-SQL Server offers various types of indexes and indexing options, such as clustered indexes, non-clustered indexes, and unique indexes. This flexibility allows for tailored index design based on specific query patterns and data access requirements.

- **Query Plan Optimization:** Indexes provide valuable statistics and information about data distribution, which the query optimizer uses to generate efficient query plans. By analyzing index usage and performance, you can optimize queries and improve the overall execution plan.

By leveraging indexes effectively, you can achieve faster data retrieval, optimized query performance, reduced I/O operations, improved concurrency, and enhanced overall database efficiency in MS-SQL Server. However, it is essential to carefully design and maintain indexes based on the specific workload and usage patterns to ensure maximum benefits.

SQL Server Clustered Indexes

A clustered index in MS-SQL Server is a type of index that determines the physical order of data within a table.

- Unlike non-clustered indexes, which create a separate structure pointing to the data, a clustered index defines the actual physical order of rows in the table based on the indexed column(s).
- By default, a primary key constraint in MS-SQL Server creates a clustered index on the specified column(s). This ensures that the primary key values define the physical order of the data. However, it is also possible to create a non-clustered primary key constraint.

- **Index Seek vs. Index Scan**: A clustered index enables **index seek** operations, which can efficiently locate specific rows based on the indexed column(s). When querying data using the indexed column(s), the database engine can navigate directly to the appropriate location within the clustered index, resulting in faster data retrieval. However, range scans or full scans of a clustered index can be slower than scanning a heap (a table without a clustered index).

- **Table Reorganization:** Changing the definition of a clustered index (e.g., dropping or adding a column) can require significant data movement and can be time-consuming for large tables. Therefore, altering the structure of a clustered index should be planned carefully to minimize disruptions.

- **Disk Space Usage:** Clustered indexes require disk space for the index structure itself, as well as the underlying data pages. If the table has non-clustered indexes, they also occupy additional disk space. Careful consideration should be given to the size and number of indexes to optimize disk space utilization.

Clustered indexes are powerful for optimizing data retrieval based on the indexed column(s) and establishing the physical order of data within a table. However, it is essential to choose the appropriate columns for indexing and perform regular maintenance to manage fragmentation and ensure optimal performance.

- **Setting up the table**

```
CREATE TABLE parts(
    part_id    INT NOT NULL,
    part_name VARCHAR(100)
);
```

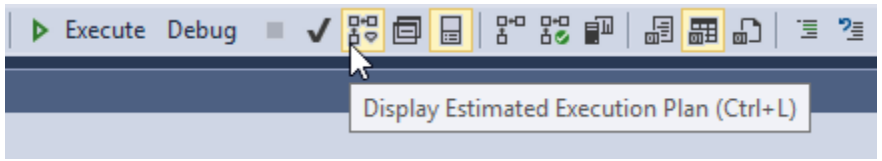
```
INSERT INTO
    parts(part_id, part_name)
VALUES
    (1,'Frame'),
    (2,'Head Tube'),
    (3,'Handlebar Grip'),
    (4,'Shock Absorber'),
    (5,'Fork');
```

The **parts** table does not have a primary key. Therefore SQL Server stores its rows in an unordered structure called a heap.

When you query data from the production.parts table, the query optimizer needs to scan the whole table to search.

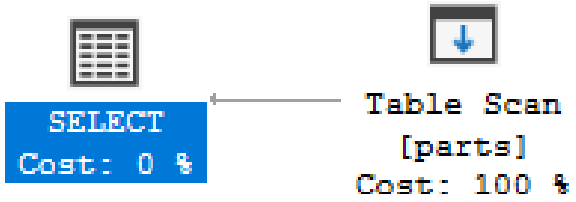
```
SELECT
    part_id,
    part_name
FROM
    parts
WHERE
    part_id = 5;
```

part_id	part_name
5	Fork



the estimated execution plan in SQL Server Management Studio

```
Query 1: Query cost (relative to the batch): 100%
SELECT part_id, part_name FROM production.parts WHERE part_id = 5
```



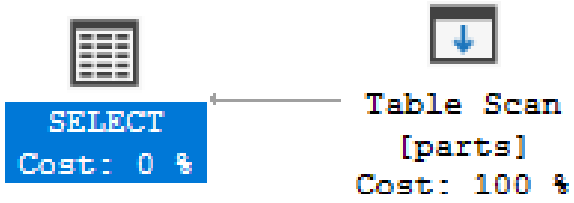
For example, this statement finds the part whose part id is 5

```
SELECT
    part_id,
    part_name
FROM
    parts
WHERE
    part_id = 5;
```

part_id	part_name
5	Fork

If you display the estimated execution plan in SQL Server Management Studio, you can see how SQL Server came up with the following query plan:

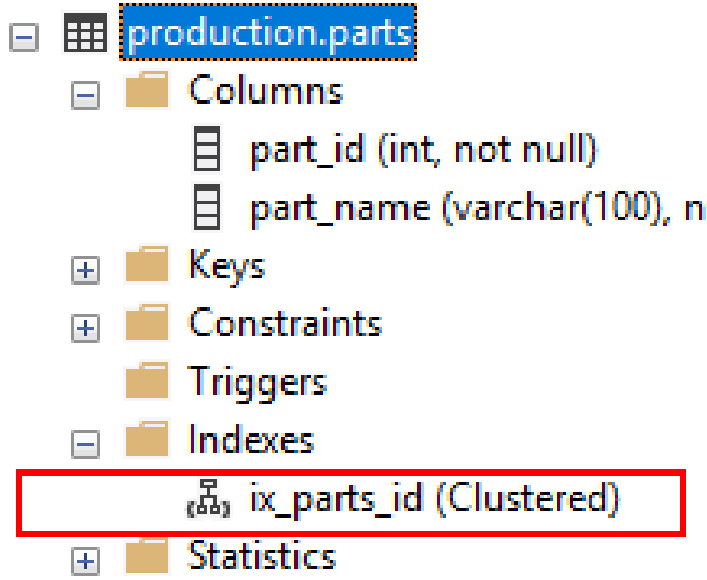
```
Query 1: Query cost (relative to the batch): 100%
SELECT part_id, part_name FROM production.parts WHERE part_id = 5
```



Using SQL Server CREATE CLUSTERED INDEX statement to create a clustered index. In case a table does not have a primary key, which is very rare, you can use the CREATE CLUSTERED INDEX statement to define a clustered index for the table.

```
CREATE CLUSTERED INDEX index_name
ON schema_name.table_name (column_list);
```

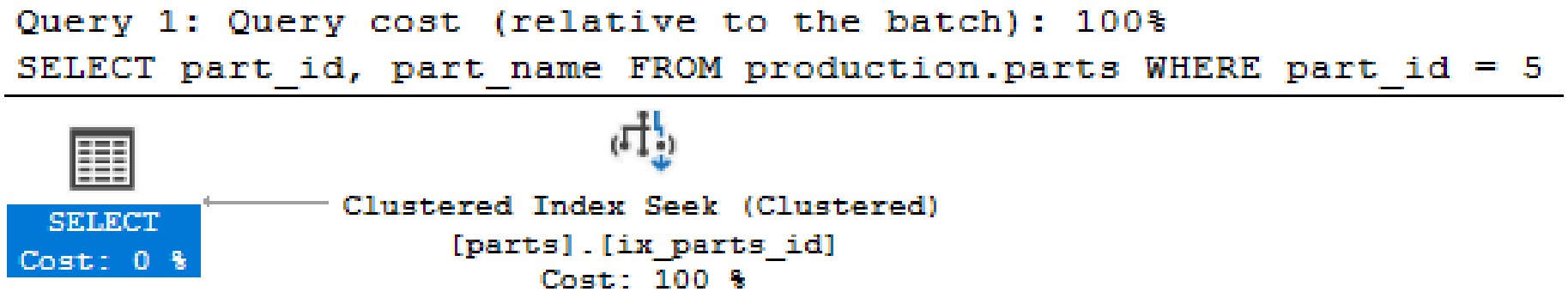
```
CREATE CLUSTERED INDEX ix_parts_id
ON parts (part_id);
```



```
SELECT
    part_id,
    part_name
FROM
    parts
WHERE
    part_id = 5;
```

part_id	part_name
5	Fork

When executing the following statement, SQL Server traverses the index (Clustered Index Seek) to locate the rows, which is faster than scanning the whole table.



SQL Server non-clustered indexes

A nonclustered index is a data structure that improves the speed of data retrieval from tables.

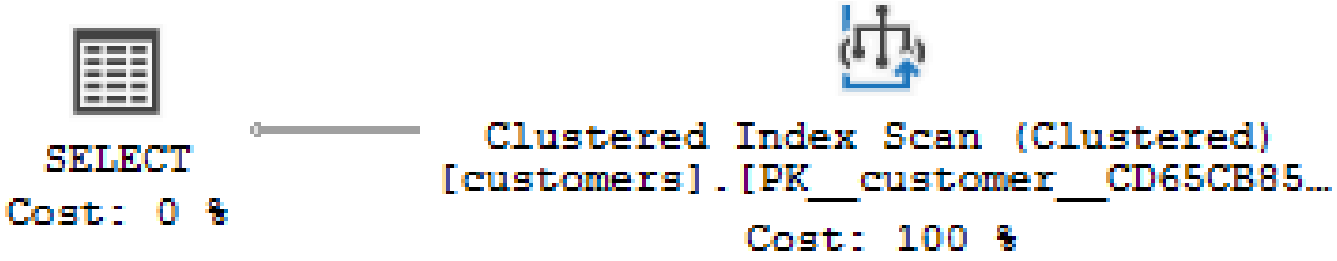
- Unlike a clustered index, a nonclustered index sorts and stores data separately from the data rows in the table.
- It is a copy of selected columns of data from a table with the links to the associated table.
- Similar to a clustered index, a nonclustered index uses the B-tree structure to organize its data.

This statement finds customers who locate in Atwater.

```
SELECT
    customer_id,
    city
FROM
    customers
WHERE
    city = 'Atwater';
```

customer_id	city
24	Atwater
151	Atwater
253	Atwater
735	Atwater
1427	Atwater

If you display the estimated execution plan, you will see that the query optimizer scans the clustered index to find the row.

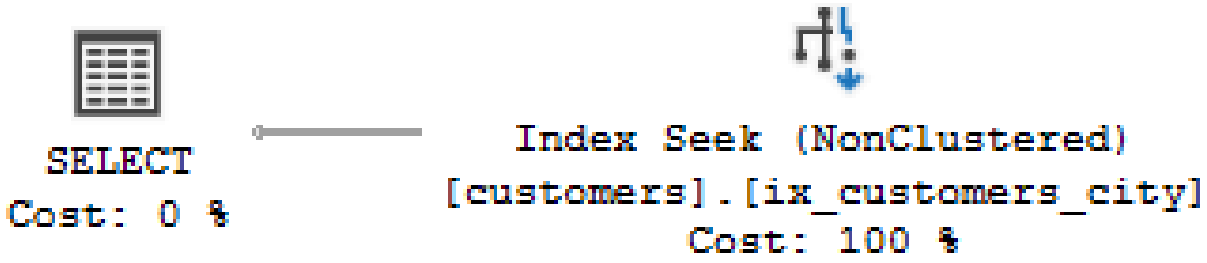


- **Creating Non-Clustered Index**

```
CREATE [NONCLUSTERED] INDEX index_name
ON table_name(column_list);
```

```
CREATE INDEX ix_customers_city
ON customers(city);
```

Now, if you display the estimated execution plan of the above query again, you will find that the query optimizer uses the nonclustered index ix_customers_city:



- **Clustered Index**
  - A Table can have ONLY 1 Clustered Index.
  - A Clustered Index always has Index Id of 0.
  - A Primary Key constraint creates a Clustered Index by default.
  - Clustered Index enforces a logical order on the rows. Rows are ordered based on Clustering Key.
  - Faster to read than non clustered as data is physically stored in index order.

- **Nonclustered Index**
  - Prior to SQL Server 2008 only 249 Nonclustered Indexes can be created. With SQL Server 2008 and above 999 Nonclustered Indexes can be created.
  - Nonclustered Indexes have Index Id > 0.
  - A Unique Key constraint created a Nonclustered Index by default.
  - Nonclustered Index does not order actual data, It only orders columns present in the
  - A table may not have any Nonclustered Indexes.
  - Quicker for insert and update operations than a clustered index.

References

https://learn.microsoft.com/en-us/sql/relational-databases/indexes/clustered-and-nonclustered-indexes-described

https://dotnettutorials.net/lesson/sql-server-indexes/

https://www.c-sharpcorner.com/UploadFile/8af593/index-in-sql-server/

https://www.geeksforgeeks.org/sql-ddl-dql-dml-dcl-tcl-commands/

https://www.javatpoint.com/index-in-sql-server

https://www.sqlshack.com/sql-index-overview-and-strategy/

https://www.tutorialsteacher.com/sqlserver/indexes